

**Working title: pySWApp - An interactive, open-source python
toolbox for processing seismic surface wave data**

N. Roser¹, I. Barone², A. Carrera², A. Flores Orozco¹

¹ *Research Unit Geophysics, Department of Geodesy and Geoinformation, TU Wien, Vienna, Austria*

² *2 Department of Geosciences, Universita degli Studi di Padova, Padova, Italy*

Received 1998 December 18; in original form 1998 November 22

Key words: Geophysical data processing; Geophysical python library; Dispersion curve picking; Surface wave analysis; Open-source library

SUMMARY

We introduce the open-source python library pySWApp, which provides a flexible and semi-interactive framework for managing and processing 2D active seismic surface wave data for dispersion curve analysis. Among classical approaches for surface-wave analysis such as the Multichannel Analysis of Surface Waves (MASW), pySWApp encompasses advanced approaches for extracting dispersion curves under laterally challenging conditions: the Multi-Offset Phase Analysis (MOPA) and the Tomographic-Like Approach (Tomo2D). We use a synthetic seismic data set to demonstrate the fundamental processing capabilities of the library and test the implementation on a real 2D data set. In particular, we explore the applicability of the implemented methods to field data collected along a laterally varying subsurface. By adhering to open-science principle, we aim to democratize access to state-of-the-art surface-wave processing and foster community-driven development toward standardized workflows for 2D surface-wave analysis.

1 INTRODUCTION

Characterizing the near-surface shear-wave velocity structure is essential for a wide range of geotechnical and geophysical applications, including site-response analysis, infrastructure assessment, environmental investigations, and urban subsurface mapping. Surface-wave methods are increasingly used for this purpose, as they rely on the analysis of surface-wave dispersion to extract dispersion curves that can be inverted to obtain shear-wave velocity profiles (e.g., Socco & Strobbia, 2004; Foti et al., 2015). Among these methods, the Multichannel Analysis of Surface Waves (MASW) has become a standard technique in active-source surveys (Park et al., 1999).

In the classical MASW workflow, a linear multichannel receiver array records surface-wave motion generated by an active source (e.g., sledgehammer or vibratory source). Typically between 12 and 96 evenly spaced geophones with a low natural frequency (4.5 Hz) are deployed, and the sources are positioned collinearly with the array. The recorded waveforms are transformed into the frequency–phase-velocity (f – c) domain using slant-stack, phase-shift, or related algorithms (Park et al., 1998; Park et al., 1999). From the resulting f – c spectrum, the fundamental and, when present, higher-mode dispersion curves are identified and inverted (e.g., via least-squares or Monte

Carlo methods) to retrieve a 1D S-wave velocity profile beneath the midpoint of the array (Park et al., 1999; Maraschini & Foti, 2010). Resolving lateral variations is limited with MASW, as the method assumes lateral homogeneity within the analysis window. To address this, the spectral analysis is often performed on spatial windows moved along the array (e.g., Pasquet & Bodet, 2017). Reducing the window length, however, improves lateral resolution at the expense of spectral resolution, so window sizes are typically chosen as a compromise between the two.

The Multi-Offset Phase Analysis (MOPA) method introduced by Strobbia & Foti (2006) offers an alternative approach to estimating dispersion curves that better accommodates lateral heterogeneity. MOPA performs a linear regression of phase versus offset at each frequency, predicting local wavenumbers (and thus phase velocities) while preserving lateral resolution even when short spatial windows are used (Strobbia & Foti, 2006; Vignoli et al., 2011). Conventional MOPA, however, assumes a dominant single mode at each frequency and excluding single modes by prior filtering may be necessary for the method to be effective.

The tomography-like inversion proposed by Barone et al. (2021), and derived from surface-wave tomography, formulates phase differences between adjacent receivers as a linear inverse problem under the assumption of straight surface-wave raypaths for each frequency. This technique yields quasi-2D phase-velocity maps that can capture sharp lateral contrasts.

Despite these methodological developments, existing open-source surface-wave processing tools - e.g., Geopsy (Wathelet et al., 2004), SWIP (Pasquet & Bodet, 2017), MASWaves (Ólafsdóttir et al., 2018; Ólafsdóttir et al., 2018), SWprocess (Vantassel & Cox, 2022)- remain largely centered on 1D analysis or offer limited flexibility (e.g., fixed workflows, restricted support for lateral-variability methods, or partially closed-source components).

We present pySWApp, an open-source Python library for active-source surface-wave analysis that integrates classical MASW, single-mode MOPA, and the tomography-like approach of Barone et al. (2021) within a semi-interactive, flexible, user-friendly, and modular framework. The library's primary function is to extract surface-wave dispersion curves with optional uncertainty estimates from multichannel active-source seismic waveform data acquired along linear profiles to obtain pseudosection of spatially varying phase velocity. The library supports various acquisition

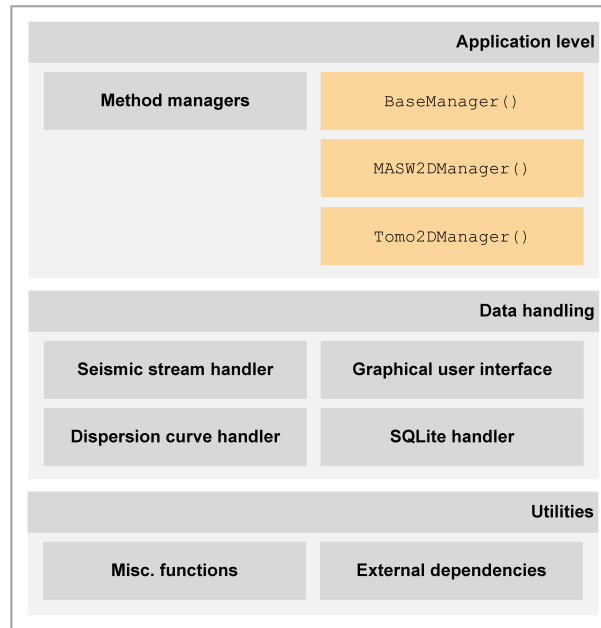


Figure 1: General architecture of the library illustrating its main components.

geometries, including off-end shots, shot repetitions and roll-along configurations. Standard processing and post-processing strategies to filter wave-field data and dispersion curves are provided for surface-wave mode isolation and quality control.

After an introduction of the library’s architecture including its core methods, the general library workflow and tailored processing classes for active 1D and 2D surface-wave analysis are presented. This is followed by a description of the SQL data handling and interactive modules for dispersion curve picking and filtering. We then showcase main capabilities of pySWApp, i.e., extraction of dispersion curve pseudosections based on different methods, with one active-source field data set.

2 LIBRARY DESIGN

The library is structured into three main components (see Fig. 1): (1) application level - providing classes tailored for processing of active 1D and 2D surface-wave analysis (SWA) data, (2) data handling level - composed of classes for handling seismic waveform and dispersion curve data, as well a graphical user interface for data viewing and specific manual processing tasks, and a SQLite

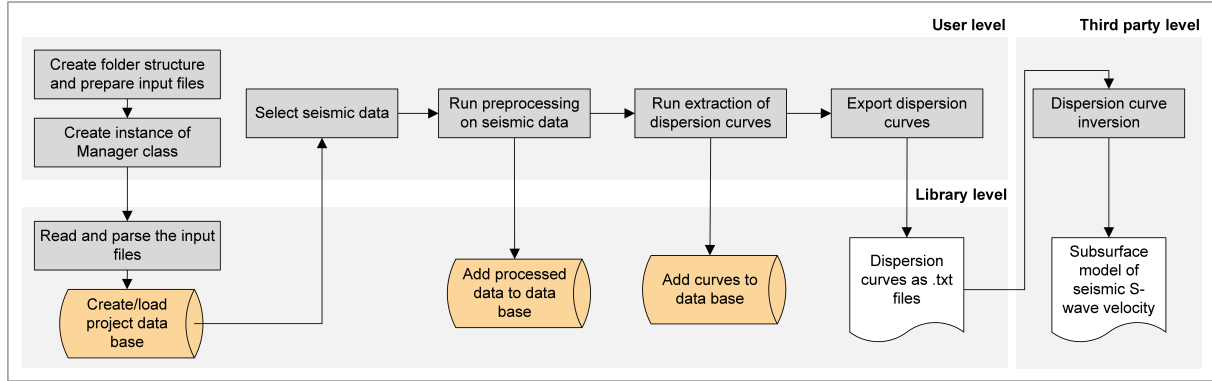


Figure 2: General library workflow for the extraction of dispersion curves from seismic waveform data.

handler for managing a SQLite database, and (3) utilities, a set of functions for import and export of files, plotting, physical relations, and solving linear mathematical systems.

The implementation builds on selected features of the recently introduced library *formikoj* (Steiner & Flores Orozco, 2023), which provides a flexible framework for managing, modelling, and processing seismic refraction data. These borrowed features include a required project directory structure, a standardized representation of field geometry, and the use of a SQLite database for storing specific inputs and outputs for flexible data processing.

2.1 Application level

The application level (user interface layer) of the pySWApp library represents the highest layer of abstraction, designed to provide an intuitive interface for end-users. It acts as the primary entry point through which users can interact with the library's functionality, abstracting away lower-level implementation details for SWA processing. The design of the application level focuses on usability, flexibility, and alignment with SWA-specific workflows. It encompasses the base class *BaseManager* for general data management and processing, upon which the two classes *MASW2DManager* and *Tomo2DManager* are built, which are tailored for processing of 2D SWA data.

2.1.1 *General library workflow*

The general library workflow is presented in Fig. 2. At user level, a new project folder is created, required input files are organized (such as the seismic shot files) and implemented methods are called to perform SWA. At the library level the project database is set up and reading, writing, manipulation of the seismic data is handled. The final output are individual dispersion curves with associated surface location, that can be inverted to obtain S-wave velocity profiles using third-party software (e.g., Geopsy).

To create a new project, first a specific directory structure is needed:

```
project directory
├── 01_data
│   └── raw
├── 02_geom
└── 03_proc
```

Analogous to the `formikoj` library, the subfolder `01_data/raw` contains the seismic shot files in `.sg2`, `.sgy`, `.dat`, or `.mseed` format. Shot files should adhere to the naming convention "Shot_<shot file number>.extension" (e.g., Shot_1001.sg2), as the system later identifies files solely by the numeric part of their names. The geometry file (`geometry.csv`), containing the survey geometry, is located in `02_geom`. This file follows the structure as described in Steiner & Flores (2023), but additionally supports multiple shots conducted at the same location, such as repeated shots or identical shot locations during roll-along measurements. If the seismic shot data includes field geometry information (i.e., source and receiver locations) that the library is able to parse, the geometry file is generated automatically during project setup. Otherwise, the survey geometry must be created manually following the structure detailed in Tables 1 and 2.

The geometry file is defined as a plain, headerless `.csv` file in which each row represents a single station in a seismic survey and contains seven columns describing the survey layout. The first three columns represent in ascending order the station x-, y-, and z-coordinates. The toolbox, however, does not consider 3D coordinates, but only the x-values interpreted as distances along a surface profile. Prior transformation of global 3D to a local 2D coordinate system is therefore

x (m)	y (m)	z (m)	Geo	Shot	1st Geo	# Geo
0.0	0.0	0.0	1	1001	1	48
1.0	0.0	0.0	1	1002;1003	1;1	48;48
...	...	...	...	...	...	...
10.0	0.0	0.0	1	-1	1	-1
11	0.0	0.0	0	1011	1	48
...	...	...	...	...	...	...
47	0.0	0.0	1	1047	1	48

Table 1: Exemplary structure of the geometry file abstracting the survey geometry.

required. The fourth column is a binary flag indicating whether a geophone is deployed at that location. The fifth column specifies the shot information by providing the numeric shot identifier if a shot was carried out at the station. The sixth and seventh columns define the receiver spread for shots by giving the index of the first active geophone and the total number of active geophones, respectively, thereby fully describing the acquisition geometry required to link shot and receiver positions within the data management framework. In particular, column six allows the reproduction of roll-along survey by adequately setting the first geophone for the consecutive segments (see also Steiner & Flores Orozco, 2023 for further reference).

In the subdirectory *03_proc*, the extracted dispersion curves will be stored as *.txt* files.

To work with the library an instance of one of manager classes is created and the path to the project directory is provided:

```
swam = BaseManager(<project_directory>)
```

Depending on the project directory contents, (i) a data preview mode will be started if no geometry file is provided and the automatic generation fails (no project will be created), (ii) a new project will be created if no project database is found in the project directory, or (iii) an existing project will be loaded if a project database exists.

If the project directory does not exist and/or does not contain the geometry file and raw data,

Column	Content	Data type	Description
1	x-coordinate	float	x-coordinate of the station (shot or geophone)
2	y-coordinate	float	y-coordinate of the station (shot or geophone)
3	z-coordinate	float	z-coordinate of the station (shot or geophone)
4	Geophone flag	int	1 if a geophone was deployed at station, 0 otherwise.
5	Shot file number (s)	str	Numerical part of the file name, e.g., 1001, if a shot was conducted at this station, -1 otherwise. If multiple shots were conducted at the same station, shot file numbers are separated by ; First active geophone of corresponding shot file.
6	First geophone	str	If multiple shots were conducted at the same station, values are separated by ;. Number of active geophones of corresponding shot file.
7	Number of geophones	str	If multiple shots were conducted at the same station, values are separated by ;.

Table 2: Description of the columns of the geometry file.

optional arguments can be provided to the manager class to create the project directory structure and copy all files to the corresponding locations:

```
swam = BaseManager(<project_directory>, path2raw = <
    path_to_raw_data>, path2geom = <path_to_geometry_file>)
```

By additionally setting the attribute `rename` to `True`, the library will attempt to rename the files to the used shot file naming convention.

Upon project initialization, the raw data and geometry file are parsed, and an SQLite database is generated and stored within the project directory. This database contains the survey geometry, global user-defined processing settings, pre-processed and transformed wavefield data, and extracted dispersion-curves, enabling data processing to be resumed at any stage. A unique label

(with 'proc1' used as the default) is assigned to the processing session, allowing multiple processing workflows to be executed and compared within the same project. Further details on the project database structure are provided in Section 2.2.1.

The library can parse seismic shot files in .sg2, .sgy, or .mseed file formats. The outputs are the project database (<database_name>.db), and the dispersion curves in .txt file format, containing the frequency in (Hz), phase velocity (in m/s) and optional data error (in m/s) stored in the subfolder 0_csv as well as the surface locations (local x-coordinates) of the extracted dispersion curves stored in the subfolder 1_geom.

2.1.2 Base Manager

The BaseManager class creates or loads a project database and initializes its structure. It provides a suite of functions for surface-wave analysis that operate on an individual seismic shot file, including data selection, pre-processing, wavefield transformation, dispersion-curve extraction, and data visualization. Building on this core functionality, processing workflows tailored to 2D SWA can be constructed, such as the implemented MASW2DManager and Tomo2DManager workflows.

2.1.2.1 Pre-processing

The preprocess method provides functions to filter the frequency content and trim the wavefield data along the time or offset axis to improve signal-to-noise ratio (SNR). To filter the frequency content, the argument type has to be set to filter. The argument by defines the type of filtering, with the following options currently implemented: frequency filtering, resampling, polarity reversal, linear move-out (LMO) correction and application of a taper window. The syntax to apply frequency filtering is shown below:

```
# Apply bandpass, lowpass or highpass frequency filtering
swam.preprocess(type = 'filter', by = 'frequency',
                 min = 5, max = 50, filter_type = 'bandpass')
```

In the code template, a bandpass filter is applied by setting the argument filter_type to 'bandpass'. The arguments min and max define the frequency range.

The seismic wavefield data can be trimmed along the time or offset axis, by setting the argument type to `trim`. It is possible to trim based on source-receiver offsets, by defining a window specified by window midpoint and length, receiver separation, selection of specific traces, removal of specific traces, time. The example code below is used to cut the recording time by defining upper and lower cut-off values in seconds (arguments `min` and `max`), i.e. 0 seconds are removed from the beginning and 0.5 seconds from the end of the record:

```
# Cut recording time
swam.preprocess(type = 'trim', by = 'time',
                min = 0, max = 0.5)
```

Traces with source-receiver offsets outside of a user-specified window can be removed as follows:

```
# Cut traces outside of offset limits
swam.preprocess(type = 'trim', by = 'offset',
                min = 5, max = 48, which = 'forward')
```

The arguments `min` and `max` hereby define the minimum and maximum accepted source-receiver offsets. With the argument `which` it is possible to control whether the forward, reverse or both source offsets should be considered during trace selection.

Further examples regarding the pre-processing options can be found in the repository.

2.1.2.2 Wave-field transformation

For manual extraction of the apparent dispersion curves, the active-source seismic wavefield data can be transformed from time-offset domain ($t-x$ domain) to frequency-velocity domain ($f-c$ domain). Two approaches are implemented, which are the widely used phase-shift transformation (Park et al., 1999) and frequency domain beamforming (FDBF, Zywicki, 1999).

The default phase-shift transform is based on frequency-domain phase alignment and stacking. After Fourier transforming each trace, the method applies a phase-shift operator (i.e., plane-wave steering vector)

$$w = e^{i2\pi fx/c} \quad (1)$$

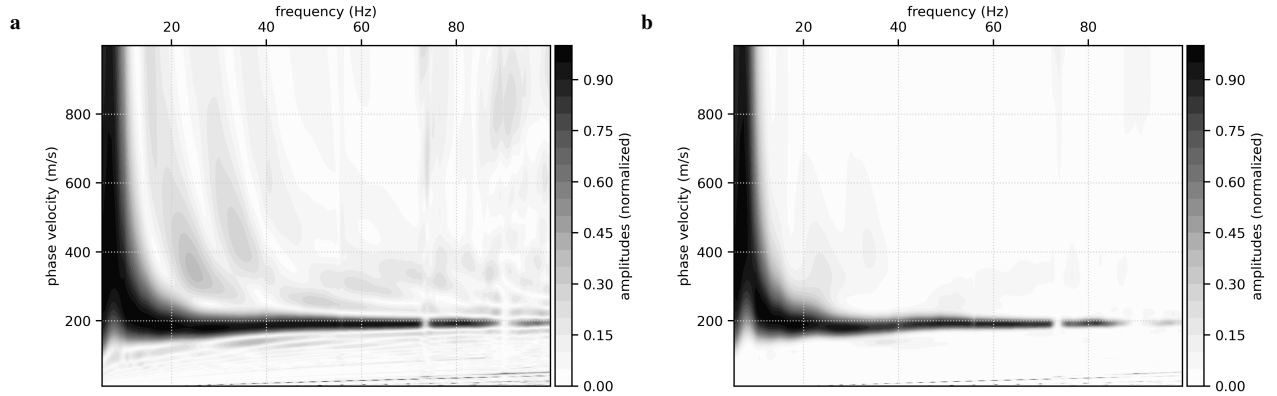


Figure 3: a) Phaseshift transform. b) FDBF transform.

where f is the frequency, x the offset and c a trial phase velocity. The steering vector compensates for the propagation delay expected for a Rayleigh wave travelling at the trial phase velocity c . For each frequency, traces are shifted accordingly and summed. Constructive interference occurs only when the trial phase velocity matches the true phase velocity, producing amplitude maxima along the apparent dispersion curves.

The FDBF is performed with a cylindrical steering vector (Zywicki, 1999), which more accurately represents the geometric spreading of cylindrical Rayleigh-wave propagation. After transforming the data into the frequency domain, the method evaluates the array response for a set of trial velocities based on the cylindrical steering vector (w). The beam power

$$B(f, v) = w^H R w \quad (2)$$

is computed for each (f, c) -pair, where R is the spatio-spectral correlation matrix (Zywicki, 1999). Maxima in the power spectrum correspond to the apparent Rayleigh-wave dispersion curve.

The transformation is performed through the transform method:

```
# Phase-shift transform
swam.transform(method = 'phaseshift')

# FDBF transform
swam.transform(method = 'fdbf')
```

Exemplary $f - c$ images obtained with both transformations are shown in Fig. 3.

2.1.2.3 Dispersion curve extraction

Four options are provided to extract dispersion curves from the seismic data set: (1) manual picking in $f - c$ or $f - k$ domain, (2) extraction of the maximum amplitudes in the $f - c$ domain, (3) multi-offset phase analysis (MOPA), and (4) a tomographic-like approach.

The first option is the classical approach, which consists of transforming the data from $t - x$ to $f - c$ domain and manually picking the dispersion curves in an interactive plot window. This option is further described in Section 2.2.2. The second option is to extract the maximum amplitudes per frequency in the $f - c$ -spectrum using the `extract` method:

```
# Extract dispersion curve based on maximum amplitudes
swam.extract(method = 'max')
```

The third option is the multi-offset phase analysis (MOPA) method by Strobbia & Foti (2006), that retrieves the dispersion curve based on weighted linear regression of the phase-offset data. Using the `extract` method, MOPA can be performed with a user-defined abort criterion and phase uncertainty:

```
# Extract dispersion curve based on MOPA
swam.extract(method = 'MOPA', stopAtChi2 = 1, rel_err = 0.01)
```

The fourth option is to run the tomographic-like approach (Barone et al., 2021), which is detailed in Section 2.1.5.

To save the dispersion curves to disk, the following command is executed:

```
swam.save(method='MOPA', dc_mode=0)
```

The argument `method` specifies the approach used to extract the dispersion curves, while `dc_mode` indicates the mode of propagation (`dc_mode = 0` corresponds to saving the fundamental mode).

2.1.2.4 Post-processing of dispersion curves

The method `process_curves` can be used to post-process the extracted dispersion curves. The toolbox provides several processing options: (1) a moving average filter with a user-defined kernel size can be applied to smooth the curves, (2) the data can be resampled to a preferred frequency

range, (3) values that do not satisfy user-specified conditions can be excluded, and (4) the data uncertainty can be estimated following the dispersion-curve uncertainty method of O'Neill et al. (2003).

To apply smoothing, the following line is executed, where the argument `method` specifies the approach used to extract the dispersion curves:

```
swam.process_curves(type='smooth', 'kernel_size':3, method='
MOPA')
```

To apply resampling, the following line is executed:

```
swam.process_curves(type='resample', pmin = 25, pmax = 45, pn =
30, method='MOPA')
```

The arguments `min` and `max` specify minimum and maximum frequency, and the argument `pn` the number of points.

To apply filtering, the following line is executed:

```
swam.process_curves(type='filter', pmin = 25, pmax = 45, param
= 'frequency', method='MOPA')
```

The arguments `min` and `max` specify minimum and maximum accepted values, and the argument `param` to which parameter the filtering should be applied (can be set to `frequency` or `velocity`).

To perform the uncertainty estimation, the following line is executed:

```
swam.process_curves(type='estimate_error', method='MOPA')
```

Figure 4 illustrates the resulting dispersion curves after applying the various processing operations.

2.1.2.5 Visualization

Individual plots can be created by calling the method `plot` and specifying the type of plot. Per default the seismic waveforms as wiggle trace is visualized by executing the following line (see Fig. 5:

```
swam.plot()
```

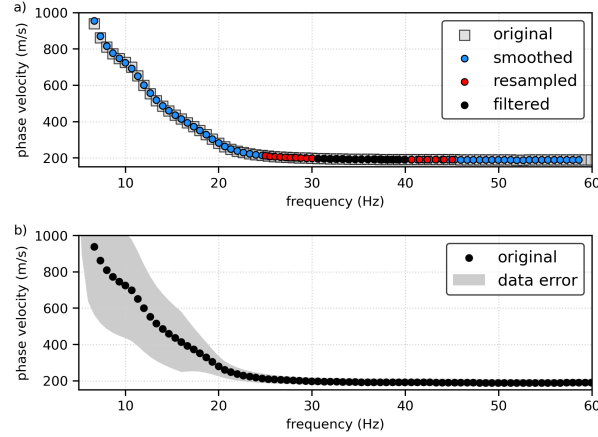


Figure 4: a) Application of different processing methods to the dispersion-curve data. b) Dispersion-curve uncertainty estimation after O'Neill et al. (2003).

The field geometry is for instance displayed by setting the type to 'geom' (see Fig. 6):

```
swam.plot('geom')
```

2.1.3 2D Multichannel-analysis of surface waves workflow

The MASW2DManager class is designed to apply the library's functionalities to a 2D active-source dataset and to provide methods to perform the classical MASW workflow, with optional spatial windowing and curve combination for uncertainty estimation (Fig. 7).

The classical MASW approach provides a robust estimation of the $f - c$ spectrum, as multiple channels are processed simultaneously, allowing effective suppression of noise, body waves, and other unwanted wavefields (Park et al., 1999; Xia et al., 1999; Liu et al., 2024). MASW can be used to retrieve fundamental and higher modes when more than one mode of propagation dominates, and the data quality and acquisition geometry are adequate. However, the method inherently as-

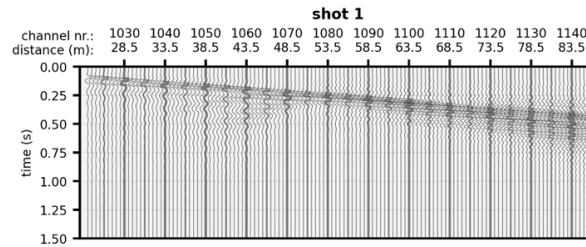


Figure 5: Plot of the seismogram.

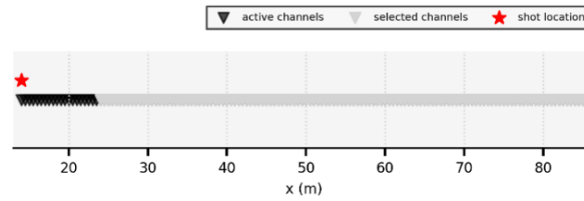


Figure 6: Plot of the field geometry.

sumes lateral homogeneity beneath the geophone spread. Strong lateral variations, such as faults, abrupt lithological boundaries, or voids, violate this assumption and may lead to misinterpretation or spatial gaps between dispersion curves along a profile (Lin, 2014). Small-scale lateral heterogeneities are often smoothed out due to the spatial windowing and the need for dense receiver arrays over a sufficiently long offset range (Park et al., 2007). MASW is subject to the trade-off between depth of investigation and lateral resolution: larger spreads and lower frequencies increase penetration depth but degrade spatial resolution, whereas shorter arrays improve lateral resolution at the cost of shallower investigation (Foti et al., 2014). Mode identification (fundamental versus higher modes) typically requires manual interpretation of the spectral images, which can be subjective and error-prone, especially in noisy environments or when strong multi-mode interference is present.

To initiate the MASW workflow, an instance of the `MASW2DManager` class is created as follows:

```
swam = MASW2DManager(<project_directory>)
```

All previously introduced methods can then be applied to the entire 2D dataset. A spatial window can be defined by calling the `moving_window` method:

```
swam.moving_window(minoffset=5, maxoffset=20, wlen=24, wmove=1)
```

Here, `minoffset` and `maxoffset` specify the permitted near-offset range for the windows, i.e., the minimum and maximum distances (in meters) between the source and the first receiver closest to the source to include in the window selection. The argument `wlen` defines the window length in terms of the number of traces, while `wmove` specifies the increment by which the window is moved along the acquisition array (`wmove = 1` corresponds to shifting the window by one trace).

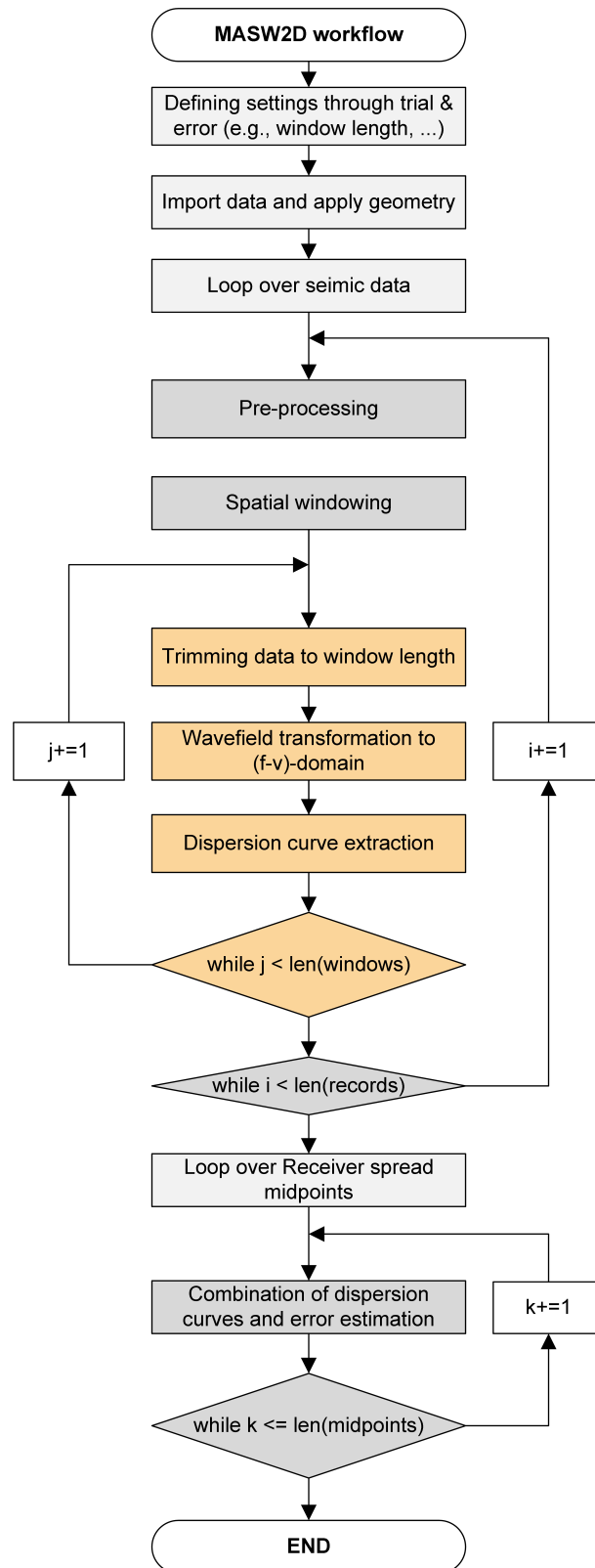


Figure 7: General MASW2D workflow with spatial windowing.

The pseudocode below demonstrates how the MASW workflow can be performed, which yields the phase-velocity pseudosection in Fig. 15(a):

```
# Create instance of class
swam = MASW2DManager(< project_directory >)

# Retrieve subsets from data
swam.moving_window(minoffset=5, maxoffset=20, wlen=24, wmove=1)

# Wave-field transformation
swam.transform('fdbf')

# Extract dispersion curves by manual picking
swam.gui_interact('pick')

# Plot pseudosection
swam.plot('pseudosection', method = 'fdbf')
```

Curve combination can be used subsequently to merge dispersion curves corresponding to the same receiver spread midpoint. This can be achieved either by resampling all curves to a common frequency range and averaging the data, or by applying the wavelength-dependent binning approach proposed by Ólafsdóttir et al. (2018) as shown in (see Fig. 9). In addition to producing an average dispersion curve for each location, the procedure allows estimation of the associated data uncertainty. To perform curve combination the method `combine` is used:

```
swam.combine(method='fdbf', filter=False, **kwargs)
```

where the argument `method` denotes the dispersion curve extraction method. All dispersion curves for a given midpoint can also be manually filtered prior to combination by setting the argument `filter` to `True`.

2.1.4 2D Multi-Offset Phase Analysis workflow

Within the `MASW2DManager` class, instead of extracting the dispersion curves from spectral images, it is also possible to run the the Multi-Offset Phase Analysis (MOPA; Strobbia & Foti, 2006)

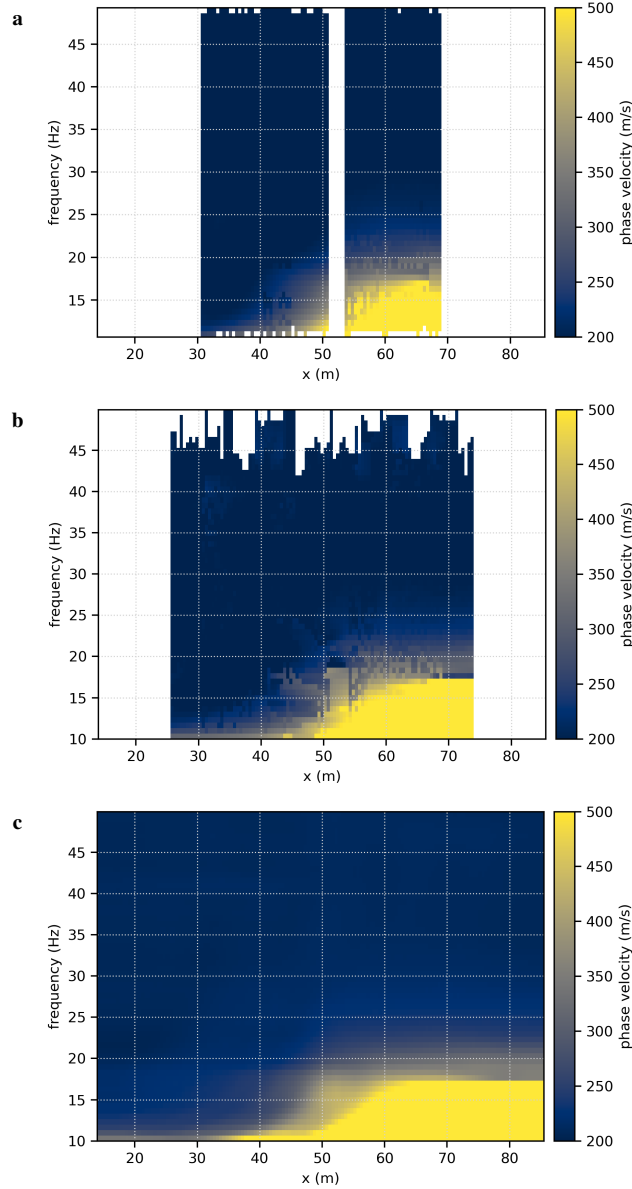


Figure 8: a) Phase velocity pseudosection after running the MASW workflow with manual dispersion curve picking. b) Pseudosection after extracting the dispersion curves in each window using MOPA. c) Phase velocity pseudosection after running the Tomo2D workflow.

workflow on the 2D dataset including the spatial windows by calling the extract method. The method will determine internally whether windows exist, that should be considered.

In contrast to classical MASW, the MOPA method offers improved lateral resolution and enhanced sensitivity to spatial variability. Although it still assumes locally 1-D behavior within each analysis window, MOPA can be applied to smaller subsets or sliding windows, thereby reducing spatial averaging and improving lateral continuity along the profile (Strobbia & Foti, 2006; Barone

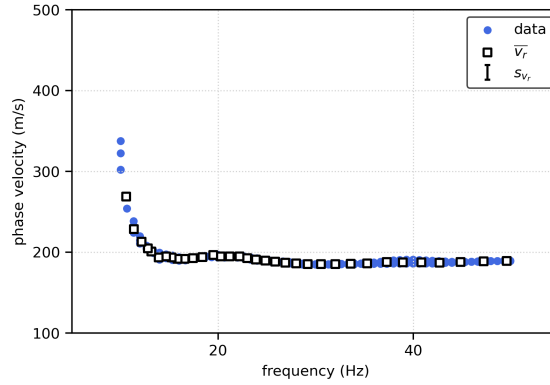


Figure 9: Combined dispersion curve based on the binning procedure of Ólafsdóttir et al. (2018).

et al., 2021). MOPA provides a more objective extraction of dispersion information by using linear regression of phase versus offset, enabling a formal quantification of uncertainty. A limitation of the standard formulation is its reduced reliability in the presence of multiple overlapping modes, unless additional modal separation techniques (e.g., $f - k$ filtering) are applied. Recent developments in multimode MOPA (MMMOPA) allow simultaneous retrieval of the fundamental and first higher modes (Barone et al., 2020).

Following Strobbia & Foti (2006), the MOPA method consists of the following steps:

1. Fourier-transform to $f - x$ domain
2. Computation of phase (ϕ) per offset (x) for each frequency component (f)
- 3a. Weighted linear regression of ϕ - x data to obtain the wavenumber (k)
- 3b. Convergence test: if $\chi^2 > 1 \rightarrow$ jump to step 3c, otherwise jump to step 4
- 3c. Near-field effect: Muting receivers too close to the source and jump to 3a
4. Computation of phase velocity based on: $c = 2\pi f / k$

The abort criterion is defined as $\chi^2 = \sum_i (\frac{\phi_{i,d} - \phi_{i,m}}{\sigma_i})^2$, where ϕ_d are the experimental phase, ϕ_m the predicted phases, and σ the phase uncertainties.

The pseudocode below demonstrates how the MOPA workflow can be performed, which yields the phase-velocity pseudosection in Fig. 15(b):

```

# Create instance of class
swam = MASW2DManager(< project_directory >)

# Retrieve subsets from data
swam.moving_window(minoffset=5, maxoffset=20, wlen=24, wmove=1)

# Extract dispersion curves using MOPA
swam.extract(method = 'MOPA', stopAtChi2 = 1, rel_err = 0.01)

# Plot pseudosection
swam.plot('pseudosection', method = 'MOPA')

```

2.1.5 2D Tomographic-like approach workflow

The Tomo2DManager class is designed to apply the tomographic-like approach proposed by Barone et al. (2021) to the SWA data set. Instead of picking dispersion curves per shot or per window, the tomographic-like approach, which we will refer to as Tomo2D hereinafter, reduces surface-wave dispersion tomography to a linear tomographic inversion per frequency. By assuming straight ray propagation, the forward problem for a given frequency becomes

$$\Delta\phi = k\Delta x \quad (3)$$

where $\Delta\phi$ is the phase difference between adjacent receivers along the profile, k is the wavenumber and Δx the separation between the receivers. This can be written as least-squares problem with additional regularization

$$\begin{bmatrix} W_d \mathbf{A} \\ \lambda W_m \end{bmatrix} \mathbf{m} = \begin{bmatrix} W_d \mathbf{d} \\ 0 \end{bmatrix} \quad (4)$$

Here, $\mathbf{d}$ denotes the data vector, i.e., the concatenation of the m observed phase differences. The vector $\mathbf{m}$ contains the n unknown model parameters representing the local wavenumber in each spatial cell along the profile. The matrix $\mathbf{A}$ is the $m \times n$ forward operator that encodes the effec-

tive ray-path lengths (i.e., the receiver separations). The matrix W_d is the data covariance matrix, with the uncertainties of the phase-difference measurements along its diagonal, and W_m is the regularization operator used to impose smoothness. The scalar parameter λ controls the strength of the regularization. From the solution to the inverse problem (i.e, the local wavenumbers) the phase velocities can be calculated. The final result of the Tomo2D approach is then a continuous pseudosection of phase velocities as a function of both space (along the profile) and frequency (or wave length, see Fig. 15c).

Compared with MASW or MOPA, the Tomo2D method provides substantially higher spatial continuity. Instead of producing dispersion information only at discrete window centers, Tomo2D yields a continuous, gap-free 2D distribution of phase velocities along the entire acquisition line (Barone et al., 2021). Because the inverse problem explicitly incorporates phase differences between adjacent receivers, the method is able to capture both gradual velocity variations and sharper structural features. A further advantage is the no need for arbitrary spatial windowing and manual dispersion-curve picking. By formulating the problem as a regularized least-squares inversion rather than relying on subjective spectral interpretation, reproducibility of the pseudosection is higher.

As with single-mode MOPA, however, Tomo2D still relies on the assumption that a single dominant Rayleigh mode has been isolated prior to inversion. If higher modes are present but not adequately separated, the inversion may yield biased or erroneous phase-velocity estimates—particularly at depth or in complex layered environments where modal interference is significant. Moreover, the spatial inversion is inherently underdetermined, requiring regularization to constrain the solution. Appropriate smoothing strength must therefore be selected to balance the suppression of artifacts against oversmoothing subsurface variability (Aster et al., 2018).

The pseudocode below summarizes the complete Tomo2D workflow illustrated in Fig. 15(c):

```
# Create instance of class
swam = Tomo2DManager(< project_directory >)

# Retrieve subsets from data
swam.prepare_streams(min_offset=5,max_offset=1e6,min_rec=12)
```

```

394 # Compute the phase differences of the processed data
395 swam.compute_phasediff()
396 # Run Tomo2D
397 swam.run(lam=10,min_offset=5,max_offset=1e6,rel_err=0.01)
398 # Plot final pseudosection
399 swam.plot('pseudosection', method = 'tomo2D')

```

400 The workflow is organized into three main methods:

401 (i) `prepare_streams`: extracts subsets of the data while excluding receivers affected by near-
 402 field effects (`offset < min_offset`) or far-field effects (`offset > max_offset`). Essentially, the
 403 function splits each shot gather into forward (positive offsets) and reverse (negative offsets) traces
 404 based on the user defined input arguments. The argument `min_rec` specifies the minimum number
 405 of receivers required for a valid subset.

406 (ii) `compute_phasediff`: computes phase differences between adjacent receivers for all sub-
 407 sets and estimates the data uncertainty automatically when repeated measurements are available.

408 (iii) `run`: solves the linear system of equations (Eq. 2.1.5) and converts the estimated wavenum-
 409 bers into phase velocities. This method takes as arguments the regularization strength (`lam`), as
 410 well as `min_offset` and `max_offset`, which determine which columns of the matrix **A** are pop-
 411 ulated. An optional relative error value (`rel_err`) can be provided when repeated measurements
 412 are not available to estimate the variance of the phase differences.

413 **2.2 Data handling level**

414 *2.2.1 SQLite database*

415 The SQLite database stores the survey geometry, global user-defined processing settings, pre-
 416 processed and transformed wavefield data, and extracted dispersion curves. The database schema
 417 and the relationships between its entities are illustrated in Fig. 11. Using the `formikoj` library
 418 (Steiner & Flores Orozco, 2023), upon project creation, the survey geometry information is read
 419 from the geometry file and recorded in the geometry table, with stations (sources and/or receivers)

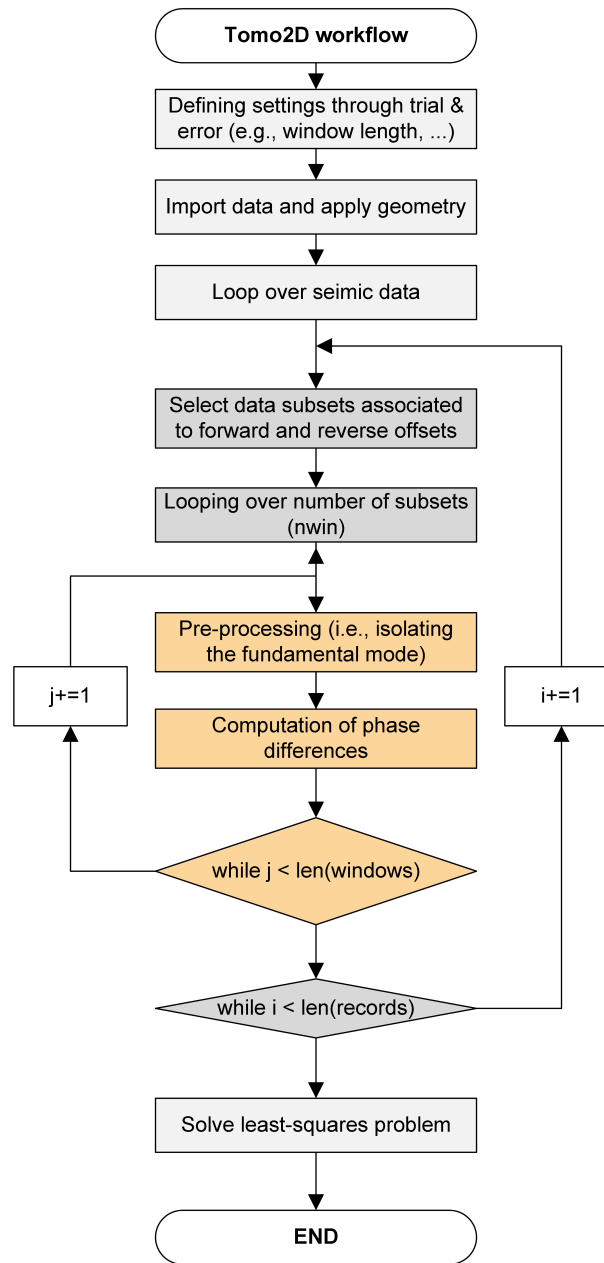


Figure 10: General Tomo2D workflow.

assigned consecutive numbers. Each unique source location is identified by a shot index number (SIN), and each unique receiver location by a receiver index number (RIN). Repeated shots are labelled with the identifier REP. The identifiers SIN, RIN, and REP are stored in separate tables and linked to the survey geometry through the station numbers.

The amplitude table contains the processed multi-trace waveform data in the time domain, while the dispersive energy obtained after wavefield transformation is stored in the dispersive_energy table. Phase differences required for the tomographic-like approach are stored in the phase_differences

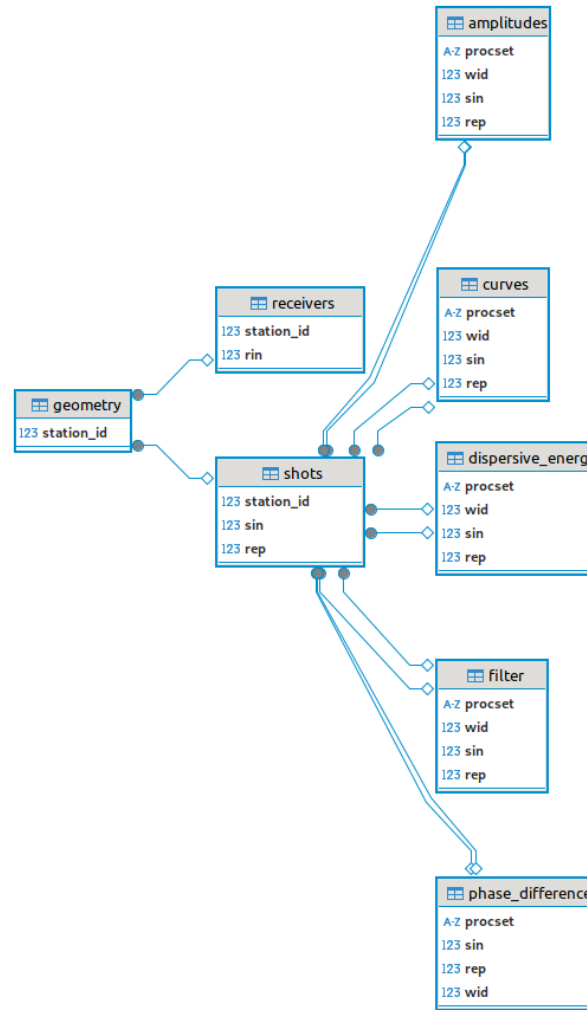


Figure 11: Entity relationship diagram.

table. Extracted dispersion curves, as well as curves resulting from the curve combination procedure are recorded in the curves table. Manually added points used to filter waveform data in the frequency–wavenumber domain are stored in the filter table. Unique datasets corresponding to a single shot location are identified by the keys SIN, REP, and WIN (for the window ID, if a spatial window was applied) as well as PROCSET, which is the unique label of the current processing session.

2.2.2 User interface

The Python library PyQt5, which provides Python bindings for the Qt toolkit, is used to create the graphical user interfaces (GUIs) in this framework. Five applications are available for (1) browsing

the data, (2) filtering the $f-k$ spectrum, (3) applying linear muting in the $t-x$ domain, (4) picking dispersion curves, and (5) filtering dispersion curves.

To browse the data, the method `gui_view` is used:

```
swam.gui_view()
```

This opens an application that allows switching between the shot gathers of the loaded data set and different visualization modes, such as waveform representations in the $t-x$ or $f-k$ domains.

Manual interaction with the data is enabled through `gui_interact`:

```
swam.gui_interact(type='filter', domain='FK')
```

Setting `type='filter'` and `domain='FK'` opens the application shown in Fig.12. In the $f-k$ spectrum, a boundary (multi-line) can be drawn to mute energy above or below it. After the filter is drawn, the 'Save' button has to be clicked to actually apply the filter and override the data in the database. Prior to saving it is possible to delete the drawn boundary using the button 'Delete'. To apply linear muting in the $t-x$ domain, the argument `domain` is set to 'TX', which opens a similar interface without the plot of the $f-k$ spectrum, where a line can be drawn to perform top or bottom muting of the amplitude data.

Manually created filter for the whole database and current procset can be saved to disk and imported to different projects using the following commands:

```
# save picked filter from database to disk
swam.save_filter(fname = <path_to_filter_location>, ftype='FK')

# import filter from disk and add to database
swam.import_filter(fname = <path_to_filter_location>)

# apply filter to data
swam.apply_filter(ftype='FK')
```

Dispersion-curve picking is performed by setting `type='pick'`, which opens the application shown in Fig.13(a-b). Instead of picking points directly on the curve, the user draws a poly-

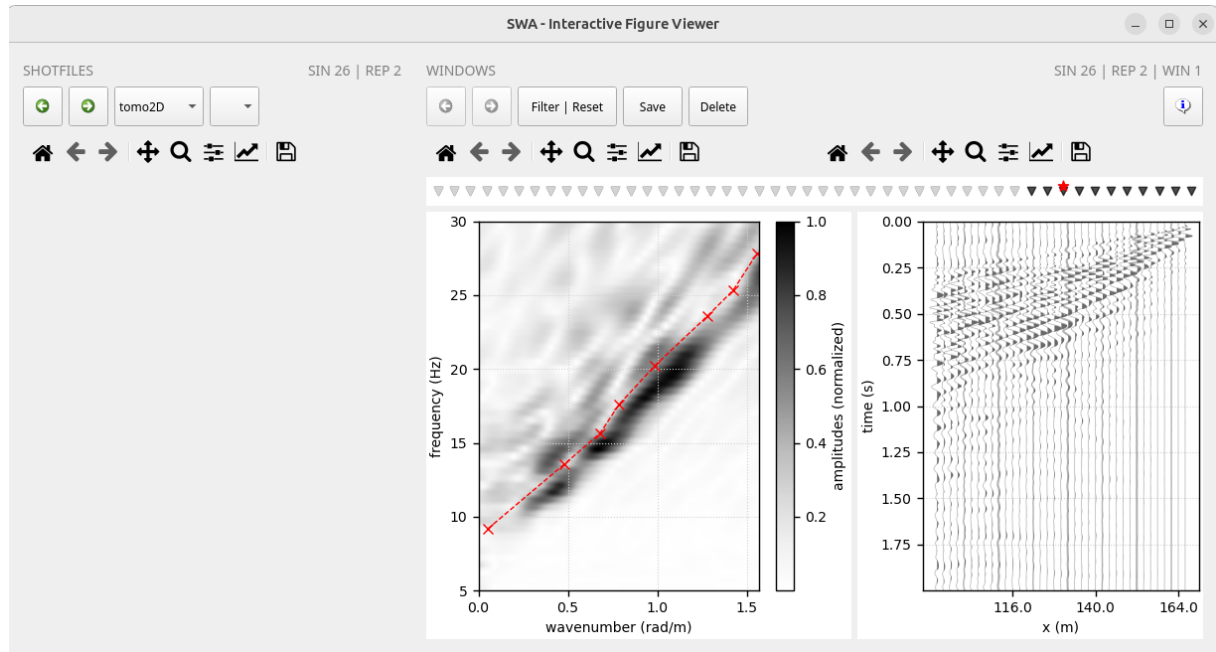


Figure 12: Graphical user interface for filtering in f - k domain.

gon around the region of maximum dispersive energy associated with a single dispersion mode (see Fig.13a). Each selected point is associated with a confidence interval computed using the uncertainty-estimation method of O'Neill (2003), resulting in frequency-dependent uncertainty that increases toward lower frequencies. Phase velocities are then extracted for each frequency within the polygon at the locations of highest amplitude. Multiple modes can be extracted by assigning an integer label (0–9) to each curve using keyboard commands (see Fig.13b). By default, picking is performed in the $f - c$ domain using one of the two integrated transforms. If both transforms have been computed, the user can switch between them via a drop-down menu, which also allows switching to the $f - k$ domain as well for picking.

Filtering of dispersion-curve points is carried out through the curve-combination method combine by setting `filter=True`. This opens the application shown in Fig.14, where polygons can be drawn to mark points for removal. These points are discarded once the application is closed.

```
swam.combine(method='fdbf', filter=True, **kwargs)
```

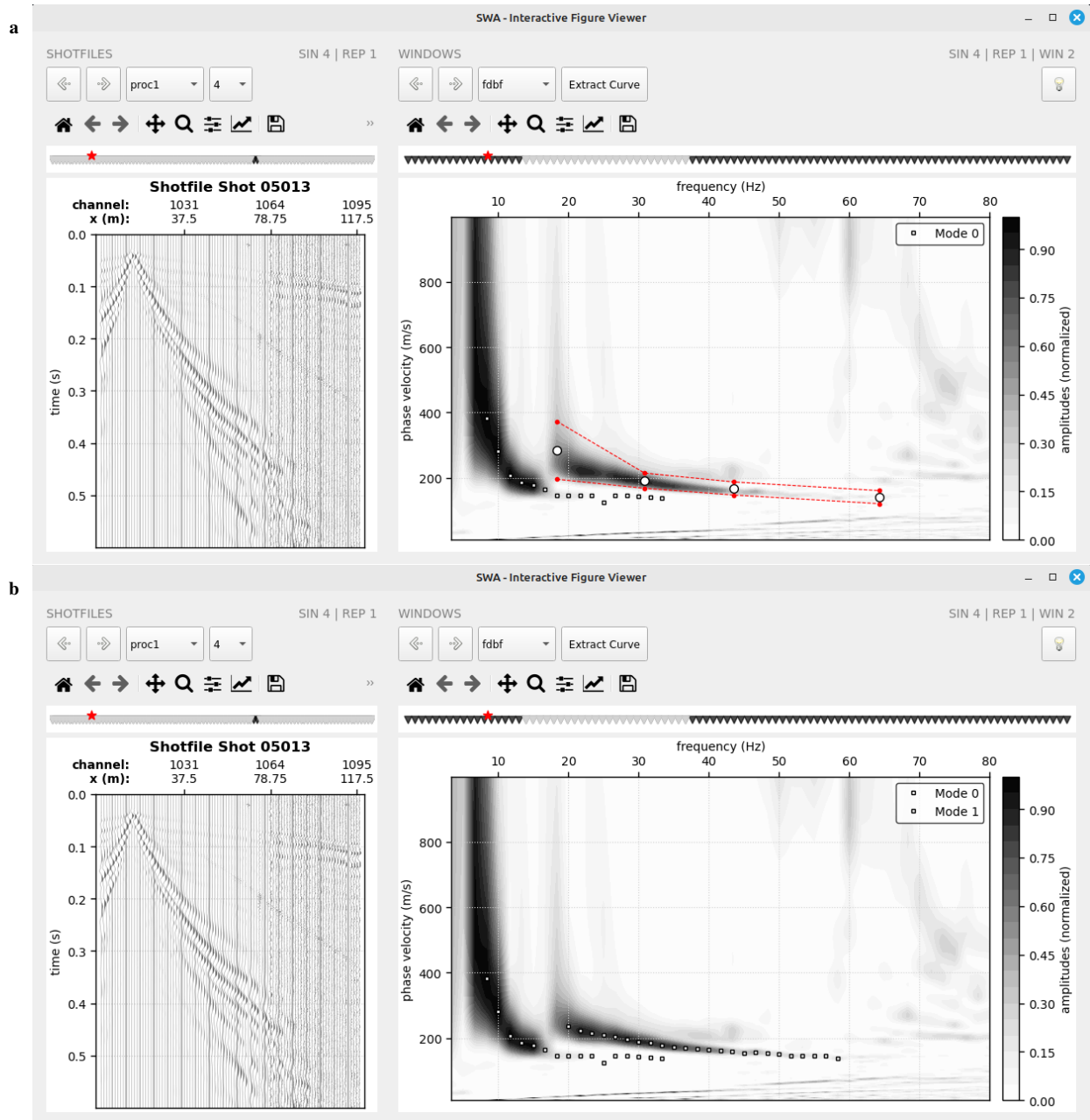


Figure 13: Graphical user interface for dispersion curve picking. a) Drawn polygon to select maximum amplitudes within. b) Extraction of multiple modes.

3 EXEMPLARY APPLICATION

To demonstrate the applicability of the pySWApp library, the dispersion curve pseudosections obtained using classical MASW approaches (manual picking and automatic maximum-amplitude picking), MOPA, and the Tomo2D method were computed for the same dataset and compared with the results reported by Barone et al. (2021).

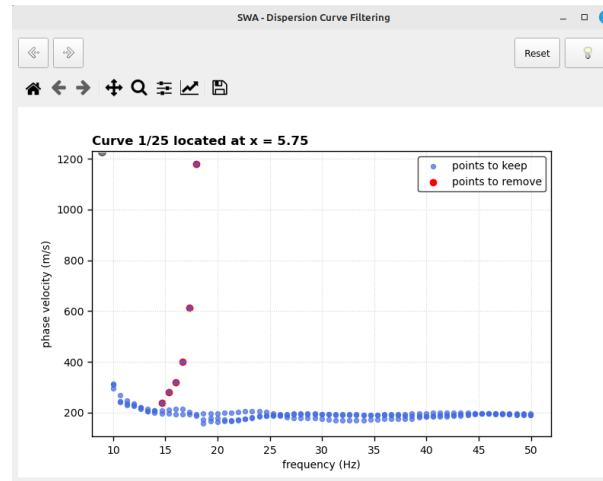


Figure 14: Graphical user interface to manually discard data points of the dispersion curve prior to dispersion-curve combination.

The field data was collected in 2019 in the Heferlbach landfill located in Mannswoth, 15 km south of Vienna, Austria, for the development and benchmarking of the tomographic-like approach. The seismic data was collected along a 272 m long line in three segments, with data acquisition conducted independently for each roll-along part. For each segment, 48 vertical geophones with a central frequency of 4.5 Hz, and separation of 2 m were deployed and shots were generated with an 8 kg sledgehammer.

Following the study by Barone et al. (2021), we applied both Tomo2D and MOPA to the entire roll-along data set and additionally compared the obtained pseudosection to the classical MASW approach based on manual and automatic picking.

First, the shot gathers were split into forward and reverse offset segments. For the MASW and MOPA methods this was done in combination with a moving spatial window:

```
swam.moving_window(minoffset = 5, maxoffset = 16, wlen = 16,
    wmove = 1)
```

As near-offset field the area between 4 to 16 m from the shot location were identified. A window length of 16 traces as trade-off between lateral and spectral resolution.

For the Tomo2D approach data was split with the tailored `prepare_streams` function, with

the minimum offset adequately set to 5 m and the minimum number of receivers of the extracted subsets limited to 12:

```
swam.prepare_streams(min_offset=5, min_rec =12)
```

Since both MOPA and Tomo2D require single-mode propagation, we first isolated the Rayleigh wave fundamental mode based on a filter manually designed in the $f - k$ -domain (see Fig. 12. Filtered $f-k$ -spectra were then back-transformed into the $x-t$ -domain. To be consistent with the results presented in Barone et al. (2021), we limited the dispersion curve analysis to the 10–25 Hz range.

Fig. 15 compares the final dispersion curve pseudosection for all four extraction options, without post-processing of the dispersion curves. Overall, the retrieved dispersion patterns show a high degree of consistency among the different processing strategies and to the results discussed in Barone et al. (2021), confirming the robustness of the fundamental dispersion features and validating the implementation of the developed library.

The classical MASW approach, based on manual picking of the dispersion image (see Fig. 15a, provides stable and interpretable dispersion trends but remains strongly dependent on operator judgment, particularly in frequency ranges where modal interference or low signal-to-noise ratios occur. The automatic maximum-amplitude picking approach reduces subjectivity and improves processing efficiency. However, it tends to preferentially track the strongest energy ridge in the dispersion image, which may lead to misidentification of modes or discontinuities where the fundamental mode is not dominant. As shown in Fig. 15(b), outliers can be present in the data, which can be further suppressed through adequate pre- or post-processing of the data.

The MOPA method enhances the spatial resolution of dispersion estimates by incorporating multi-offset phase information. This results in smoother lateral variations and improved continuity of the dispersion trends along the profile. Nevertheless, the method remains sensitive to phase unwrapping errors and to the assumption of laterally smooth velocity variations, which limits its reliability in areas with strong heterogeneity and in accordance with MASW does not sample regions close to the profile ends (see Fig. 15c).

The tomographic-like approach produces a complete dispersion pseudosection that highlights

lateral variations more clearly than the other techniques (see Fig. 15d). By integrating information from multiple receiver pairs and offsets, it provides a more spatially distributed estimate of phase velocities. However, as noted in including Barone et al. (2021), the increased spatial resolution comes at the cost of greater sensitivity to data coverage and inversion regularization parameters.

Despite these methodological differences, the overall agreement among the pseudosections confirms that all three approaches capture the main dispersion characteristics of the dataset and align with the observations in Barone et al. (2021). Variations from the results presented in their study are attributed to the subjectivity of manual filtering and picking. The observed discrepancies within the methods are consistent with the known limitations of each method and mainly affect frequency ranges where modal interference, limited offset coverage, or low signal-to-noise conditions reduce the reliability of phase velocity estimation. Consequently, the combined use of these approaches provides a useful framework for assessing the robustness of dispersion measurements and for identifying portions of the dispersion spectrum that are more reliably constrained. Thus, reliable dispersion curve data sets can be extracted and used for ongoing processing, i.e., depth inversion based on third-party software such as Dinver from the open-source Geopsy software package (Wathelet, 2008).

4 CONCLUSION AND OUTLOOK

We have introduced pySWApp, a flexible python-based processing library to extract surface wave dispersion curves from 2D active seismic shot gathers based on the classical MASW approach with manual picking and automatic picking, MOPA, and the Tomo2D method.

The capabilities, versatility and application of the framework have been demonstrated based on a synthetic and field data set. As shown, the automatization of key processing steps and tailoring of methods for 2D surface-wave analysis provides single line commands to establish high modularity and user-friendliness. The processing can be conducted purely statically or semi-interactively by performing manual picking or filtering within the implemented interactive applications.

The implementation has been tested on two major operating systems (Linux/Unix, Windows)

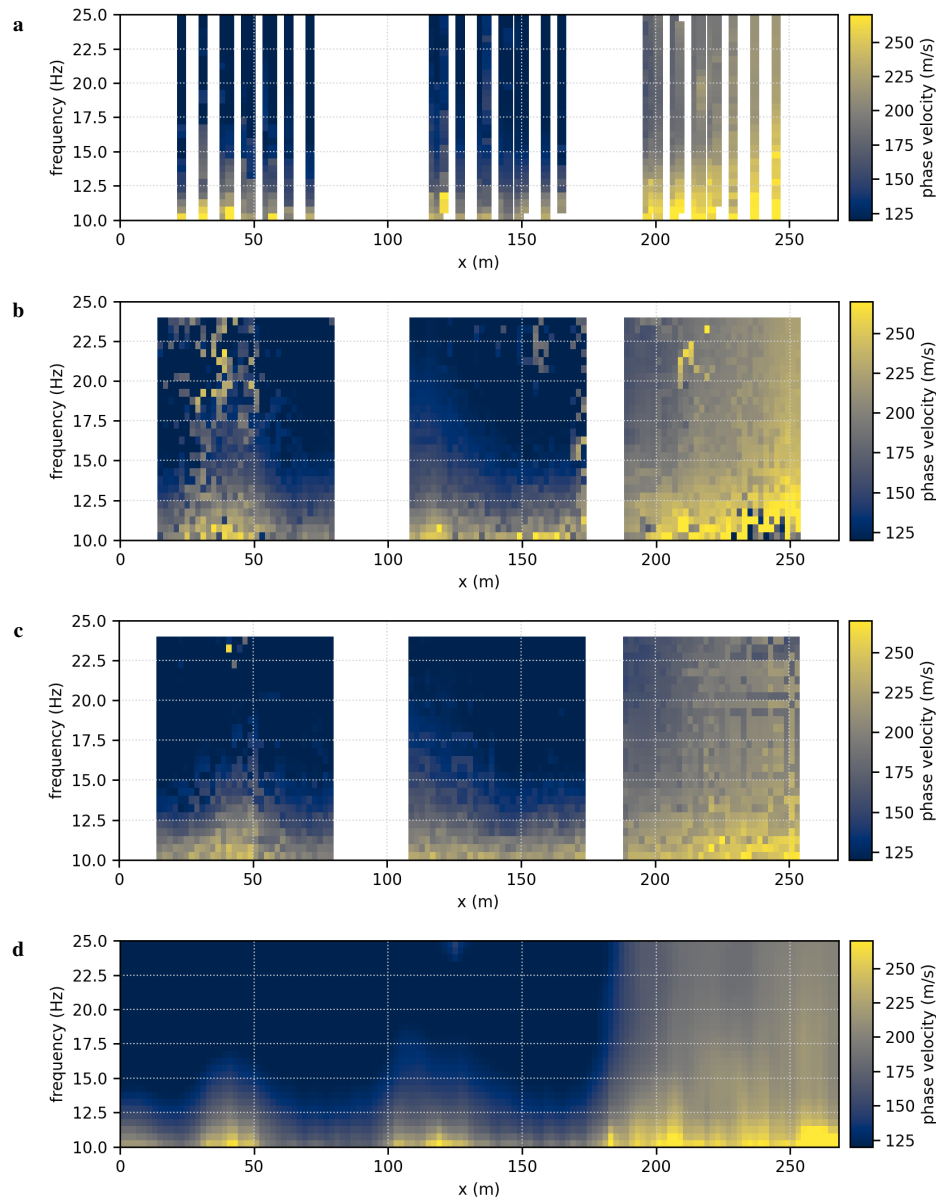


Figure 15: a) Phase velocity pseudosection after running the MASW workflow with manual dispersion curve picking. b) Pseudosection after extracting the dispersion curves in each window using MOPA. c) Phase velocity pseudosection after running the Tomo2D workflow.

via line-based execution of the commands in an ipython shell and jupyter notebook, and by executing automatized scripts.

By making the source code available under the MIT license, we aim to provide a state-of-the-art surface-wave processing library and foster community-driven development toward standardized workflows for 2D surface-wave analysis. The library is constantly under development

557 based on user input and scientific advancements to add further strategies for reliable surface-wave
558 analysis and improve processing and data storage performance.

REFERENCES

- Aster, R. C., Borchers, B., & Thurber, C. H., 2018. Parameter estimation and inverse problems. Elsevier.
- Barone, I., Strobbia, C., & Cassiani, G., 2020. Multimode multioffset phase analysis of surface waves, a new approach to extend MOPA to higher modes, *Geophysical Journal International*, 221(3), 1802-1819.
- Barone, I., Boaga, J., Carrera, A., Flores-Orozco, A., & Cassiani, G., 2021. Tackling lateral variability using surface waves: a tomography-like approach, *Surveys in Geophysics*, 42(2), 317-338.
- Foti, S., Lai, C. G., Rix, G. J., & Strobbia, C., 2014. Surface wave methods for near-surface site characterization. CRC press.
- Lin, S., 2014. Advancements in active surface wave methods: modeling, testing and inversion. Digital Repository Iowa State University, Graduate Theses and Dissertations, Paper, 13761.
- Liu, L., Tian, Y., Liu, Y., Chen, J., & Li, H., 2024. Multichannel Analysis of Ambient Noise Surface Waves Based on Semblance Phase-Shift Method, *Remote Sensing*, 16(23), 4484.
- Ólafsdóttir, E.Á., Erlingsson, S. & Bessason, B., 2018. Tool for analysis of multichannel analysis of surface waves (MASW) field data and evaluation of shear wave velocity profiles of soils, *Canadian Geotechnical Journal*, 55(2), 217–233.
- Ólafsdóttir, E.Á., Bessason, B. & Erlingsson, S., 2018. Combination of dispersion curves from MASW measurements, *Soil Dynamics and Earthquake Engineering*, 113, 473-487.
- O'Neill, A., Dentith, M., & List, R., 2003. Full-waveform P-SV reflectivity inversion of surface waves for shallow engineering applications, *Exploration Geophysics*, 34(3), 158–173.
- Park, C.B., Miller, R.D., & Xia, J., 1999. Multichannel analysis of surface waves, *Geophysics*, 64(3), 800–808.
- Park, C. B., Miller, R. D., Xia, J., & Ivanov, J., 2007. Multichannel analysis of surface waves (MASW)—active and passive methods, *The leading edge*, 26(1), 60-64.
- Pasquet, S., & Bodet, L., 2017. SWIP: An integrated workflow for surface-wave dispersion inversion and profiling, *Geophysics*, 82(6), WB47–WB61.
- Steiner, M., & Flores Orozco, A., 2023. formikoj: A flexible library for data management and processing in geophysics—Application for seismic refraction data, *Computers and Geosciences*, 176, 105339.
- Strobbia, C., & Foti, S., 2006. Multi-offset phase analysis of surface wave data (MOPA), *Journal of Applied Geophysics*, 59(4), 300-313.
- Vantassel, J.P. & Cox, B.R., 2022. SWprocess: a workflow for developing robust estimates of surface wave dispersion uncertainty, *Journal of Seismology*, 26(4), 731-756.
- Wathelet, M., Jongmans, D., & Ohrnberger, M., 2004. Surface-wave inversion using a direct search algorithm and its application to ambient vibration measurements, *Near Surface Geophysics*, 2(4), 211–221.
- Zywicki, D.J., 1999. Advanced signal processing methods applied to engineering analysis of seismic surface waves, Ph.D. dissertation, Georgia Institute of Technology.

- 594 Xia, J., Miller, R. D., & Park, C. B., 1999. Estimation of near-surface shear-wave velocity by inversion of
595 Rayleigh waves, *Geophysics*, 64(3), 691-700.